Niobium, Inc.

Dr. Cynthia Archer, Dr. Arinjita Paul, Claire Tomesch

Encrypted Network Intrusion Detection

Last Modified: Aug 4, 2026

Niobium, Inc.
444 E 2nd Street
Suite 250
Dayton OH 45402
niobium.co

Overview

The FHE hardware accelerator market is nascent, with most prospective clients still learning what privacy preservation needs they face. Growing the accelerator market to a viable level requires that Niobium provide education to those clients, along with proof that our technology of choice, Fully Homomorphic Encryption (FHE) [2], can enable solutions relevant to client business. Providing proof of FHE in relevant use cases leads to the need to develop industrially relevant *solutions* – practical, real-world FHE applications that solve client needs at realistic scale, using recognizable and domain-relevant algorithms.

This paper describes the design of one of these applications: Network Intrusion Detection (NID) based on the Kitsune [1] anomaly detection system. Network-based anomaly detection systems monitor IP connection patterns to establish baseline behavioral profiles for normal network traffic, then identify deviations that may indicate malicious activity or active cyber attacks. These systems may analyze multiple connection attributes including source and destination IP addresses, port numbers, connection duration, data transfer volumes, timing patterns, and protocol usage to build statistical models of legitimate network behavior.

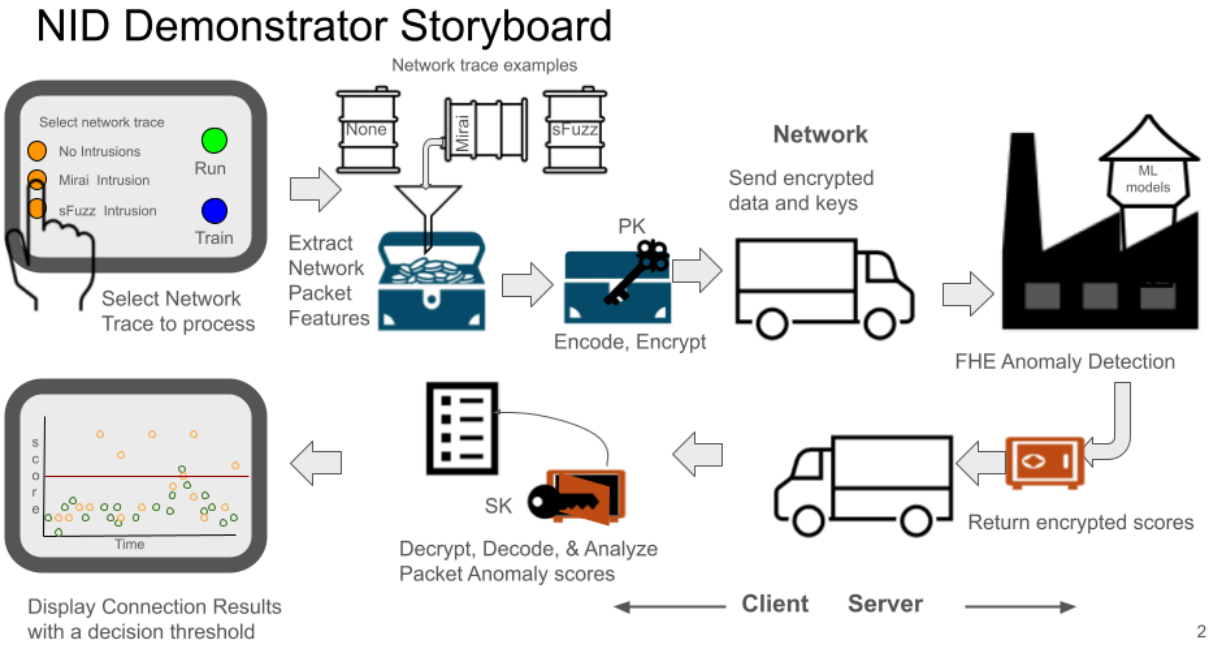
This anomaly detection framework operates by computing deviation scores for each new connection against the learned normal behavior patterns, triggering alerts when connections exceed predefined threshold values. Anomaly detection can identify various attack vectors including lateral movement within networks, data exfiltration attempts, botnet communications, and advanced persistent threat (APT) activities that often exhibit subtle but consistent behavioral anomalies compared to legitimate user traffic. Unfortunately, the same data that provides the ability to analyze connections for unusual behavior also can provide highly sensitive information to any parties that have access to it

Exacerbating these privacy concerns, NID processing is often provided by a dedicated, centralized group within an organization. While this group provides network security services to everyone in the organization, they do not want to hold private network use information, since that makes them a target for malicious activity. Our solution aims to use FHE to preserve intrusion detection provided by centralized services, while avoiding the need to *burden* those services with protecting sensitive network information.

Application Architecture

The Network Intrusion Detection application will represent a use case, wherein a client organization uses a central security organization to monitor its own network traffic for potential intrusions or other malicious activity. This use case is illustrated in the storyboard figure below.

The application will allow the operator to choose from one of several curated data sets. The selected set will be processed in the same way a client would process their own data to demonstrate the feasibility of using secure processing for network intrusion detection.



Application Storyboard

In a typical deployment, the client first establishes a connection with the server. The client generates encryption keys. The server is provided with the anomaly detection model[1] and the cryptographic context. The client assembles live network data using standard commercial tools, such as Wireshark. With an application provided by the security service, the client turns the raw pcap (or other compatible format) data of network behavior into feature vectors. These feature vectors are encoded, packed, and encrypted with an FHE compatible encryption scheme. The resulting ciphertexts, along with the relevant (public) FHE computation keys, are transmitted to

[1] Normal traffic models are developed by the client, using an application provided by the security service. This application produces an FHE compatible model trained on normal network traffic data provided by the client. The model is transmitted securely to the service provider, where it is stored for future use. A client may train and provide several models, for example: normal working hours, evening, and weekend. By having the client train the model, private network information remains with the client. The trained model provides little to no private information about network access, as it encapsulates average traffic behavior, rather than specific information about connections or accessed addresses.

the security server over an external network. Encryption protects the private nature of the network behavior in transit, as well as during processing.

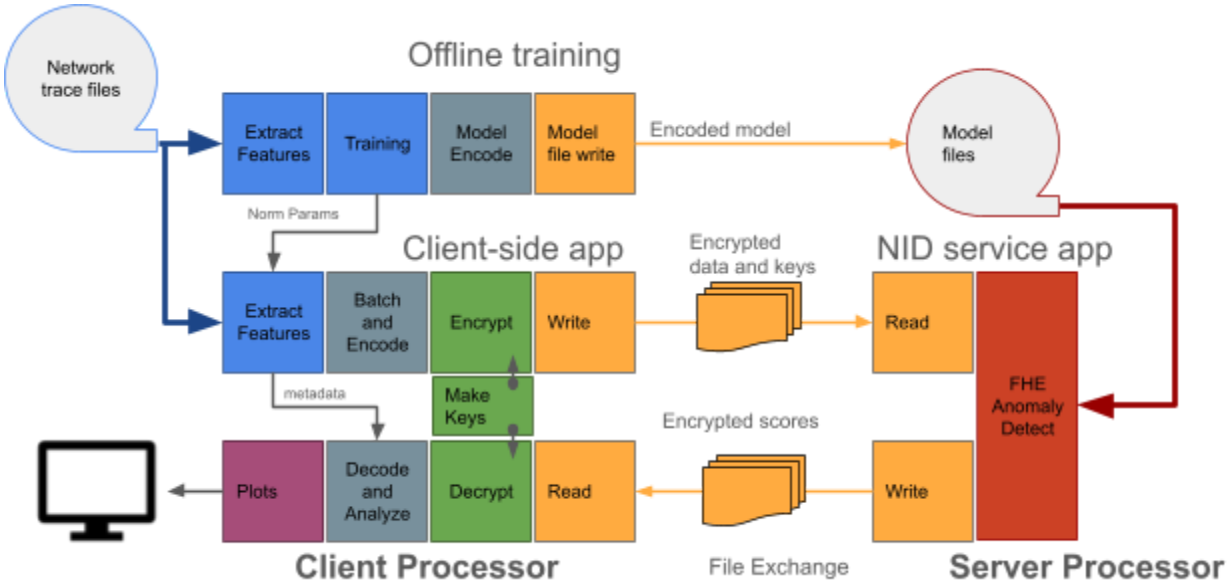
The security server accepts the encrypted data and processing keys. It then performs anomaly detection on the data, using an appropriate model from the server's library of models of normal network behavior. It returns an encrypted vector of anomaly scores, one for each packet. The client decrypts and processes these scores as desired. For instance, the client may look for time segments with high average anomaly scores, indicative of an attack. Alternatively, they may look for suspicious connections to find the addresses of potential malicious intruders.

Application Components

The NID application consists of several inter-dependent functions: client-side off-line training of normal traffic models, server-side model storage, client-side data aggregation and encryption of sample data, server-side anomaly detection, and client-side anomaly score presentation. The first two functions work together and are only used by the application developers. These are shown in the first row of components in the architecture diagram below. The last three functions also work together and are what the marketing customer will demonstrate. The complete roundtrip process is shown in the lower rows in the architecture diagram. Blue components are derived from the Kitsune NID application, grey components are new "glue" software, gold components are file transfer software, green components are FHE interface software, and the red component is the FHE anomaly detection application.

In a real-world deployment, client and server processes will run on separate machines. However, for the purposes of making this demonstrator easy to use, both are hosted on the same computer, operating as separate processes. Only the server process is FHE protected. It is structured to facilitate replacing the original CPU implementation (OpenFHE) with a faster GPGPU (general purpose graphics processing unit) implementation (FidesLib). It will also enable integrating the Niobium accelerator into the server once the hardware is ready.

Training and model storage are performed off-line. Normal network traffic models are trained on the client side, under the client's normal security procedures (not FHE). The Client-side process derives feature extraction and model training from the Kitsune application. It delivers trained models to the server-side, where they are stored for future use. For our demonstrator, these two processes are strictly for internal development. Training and model storage will take place before the application is released to the demonstration team. In a live deployment, the security service will provide a training application to the client, who will train models for the normal conditions they care about and deliver those models securely to the centralized server.



Anomaly detection is performed on live or recently gathered network traffic data. The same feature extraction module used in training converts pcap data into feature vectors. Values are normalized using parameters saved from the training phase. Feature vectors are batched and encoded for efficient ciphertext packing. The client also generates all necessary keys, both public and private. The public key is used to encrypt the batched features into ciphertexts using the OpenFHE library. The encrypted data, along with public and processing keys are delivered to the server using a file interface.

The FHE anomaly detector retrieves the appropriate model and runs the batched and encrypted feature data against that model. It produces a vector of anomaly scores, one score per packet. The encrypted scores are returned to the client. When the client receives the encrypted anomaly scores, it decrypts them and analyzes the results, turning them into plots or similarly user friendly formats.

Features are transmitted one batch of 32K packets at a time. The Client is responsible for metering delivery of batches, relieving the server of the need to temporarily store arbitrarily large amounts of ciphertext data. Initially, metering is achieved by the client waiting for a result from the server for one batch of packets before sending the next batch. Data is exchanged through the file system. More pipelined, efficient operation can be achieved by adding better buffering control later.

The client application delivers full ciphertexts representing 32K packets to the FHE server. In the moderate traffic case, this much data covers 20 minutes of network traffic for a typical user. In a heavy traffic case, it may cover only 30 to 40 seconds of traffic. Each ciphertext requires 64K x 8

Bytes x 2 x 22 levels, and there are 50 ciphertexts per batch (each one holding a single feature of all 32k packets). Thus the ciphertext load for each batch of packets is roughly 1.2GB. Assuming a typical 1 Gbps Ethernet connection from client to server, we can expect goodput of roughly 200 Mbps throughput. Consequently, a batch will take about 48 seconds to transmit over the network.

Application Script

The user will run the application using a python script. This script implements a main process with a server subprocess. The main process runs either training or the (inference) client, depending on the mode specified on the command line. When the mode is execute, the main process also spawns the server process. The default mode is execute. The data choice (Mirai, Fuzzing, Wiretap) is also specified on the command line as the activity argument. The command line will be something like:

python networkmonitor.py <activity> [-train [-pcap]]

Execution can be either in-the-clear or encrypted. Adding FHE for encrypted execution makes a substantial difference in the code. Consequently, we will not try to maintain a single code base. Instead, the in-the-clear and FHE applications will be separate branches from the code repository.

Training

For training mode, the python script will open the training file in the specified directory. When the pcap flag is present or if there is no tsv file, it will convert the pcap file to a tsv file with *tshark* and save the tsv file back into the same directory, with the “tsv” extension. The script then runs the training actions. Train uses the feature extraction class to open and process the training samples from the tsv file, one packet at a time. It then converts packet data to features. Features are input to the KitNet package, which trains the neural network. The trained model is saved to a structured binary file for later access by the server application. Normalization parameters are also trained and saved in the data directory for later use by the client application.

Execution

For execute mode, the python script will open the associated tsv file after checking that both the model and the tsv file exist. It will start the client activities and spawn the server process, with a communication pipe between them using the python subprocess package. An initial handshake occurs between the client and server processes using the communication pipe. The client generates the cryptographic context and keys, serializing context information to a file using

OpenFHE functions. It then notifies the server process that it is waiting. The server process reads and saves the context, returning a server ready message to the client.

Once the handshake is complete, the tsv file is read by the client, one packet at a time using the feature extraction class to generate feature vectors. The client process accesses the normalization parameter file and converts each incoming packet into normalized features. Packets are then grouped 32K at a time. The client applies ciphertext packing and encryption to the batched data. The resulting data is then written to a set of files accessible by the server process. Each file will contain a single feature for all 32K packets. There will be a total of 50 files per batch, one per input feature. File names will be tagged with the feature index, so the software can properly align the data with the anomaly detection model parameters.

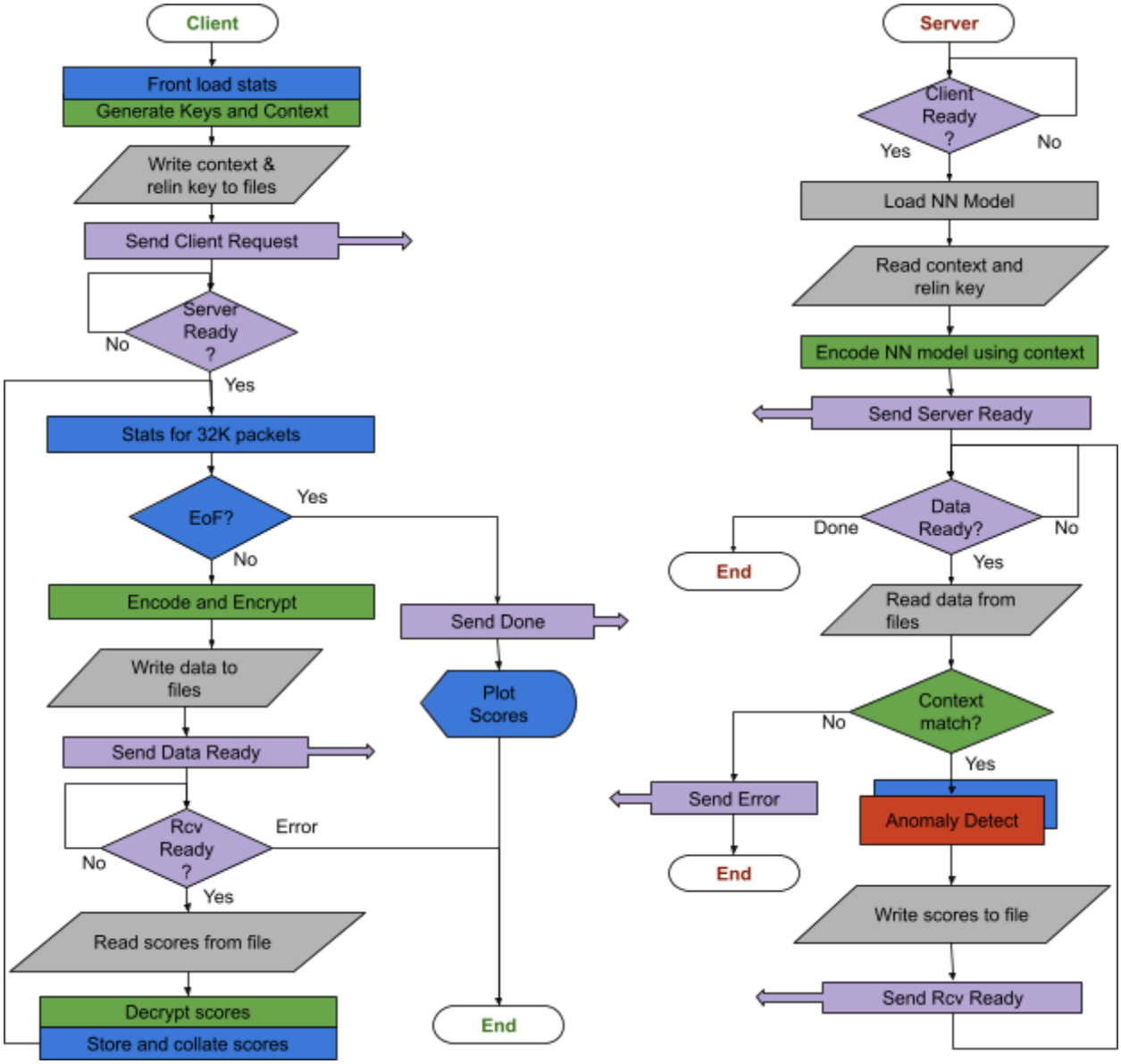
When the server process is created, it waits for a *request* from the client. The request specifies the model file. The server opens the model file and populates the KitNet model from it. During the initial handshake, it reads and saves the cryptographic context and key(s), notifying the client when it is *ready* for data. When the client process completes a batch of data, it notifies the server that a batch of data is *ready* along with the path to the data.. The server process reads in the batched data from the associated files. As long as the cryptographic context matches, it processes the feature vector through the NN model, and generates a score for each packet. When all packets are processed, it writes the scores to a file, notifies the client that scores are available, and goes back to waiting for the next batch of packet data. When the client receives a *ready* message from the server, it reads the scores file and stores it in internal memory, along with any associated meta-data (labels, connection number, protocol, etc.)

If the data context does not match the saved context, the server declares a fatal error. The server sends an error message to the client and then terminates. The client also terminates on receiving a server error message after it prints an error message to the operator.

When the client finishes reading the input tsv file, it sends a *done* message to the server, telling it to terminate. No attempt is made to process a partial batch; it is simply discarded. It waits for the final scores, which it reads and stores. The client then generates graphics using the matplotlib package. It terminates when the operator closes the plots. After the server has terminated, the parent (client) process cleans up the data transfer directory and terminates.

Flow charts for the client and server processes, with communication messages, are shown below. The client process is on the left and the server on the right. Blue blocks will be derived from the Kitsune software. Grey blocks are new “glue” software. The lavender blocks are new communications software, which will use the python process package to create a pipe to the server subprocess. The green blocks are new OpenFHE operations related to encryption, decryption, and ciphertext generation. They will be pass-through blocks for the in-the-clear application. The orange block is the new OpenFHE enabled anomaly detection. This will be

based on the KitNet software, but translated for FHE. A minimally modified KitNet execute function will be used for in-the-clear anomaly detection.



This design does not use threading to maximize throughput. This was intentional, so that we can focus on getting the FHE implementation correct before optimizing performance. However, adding threading is planned for future development.

Core Component Designs

The client and server processes of the NID demonstrator include several software components, including:

- Data file system,
- Feature extraction module,
- KitNet neural network model,
- Batched data module, and
- Cryptographic Context module.

The training process (on client-side) accesses the file system, uses the feature extraction and KitNet modules, and creates the KitNet Anomaly Detection model. The Client process (execution) also accesses the pcap files and uses the same feature extraction module. In addition it uses the batched data and cryptographic modules. This process also includes a module for generating the output graphics. The Server process (execution) uses the KitNet model and the batched data module. To save project development time, we use python for the software language of the client. That seems acceptable, because FHE execution on the server will dominate runtime. However, the server side software will be converted to C++ to maximize server-side performance and for compatibility with OpenFHE and FIDESLib.

Data File system

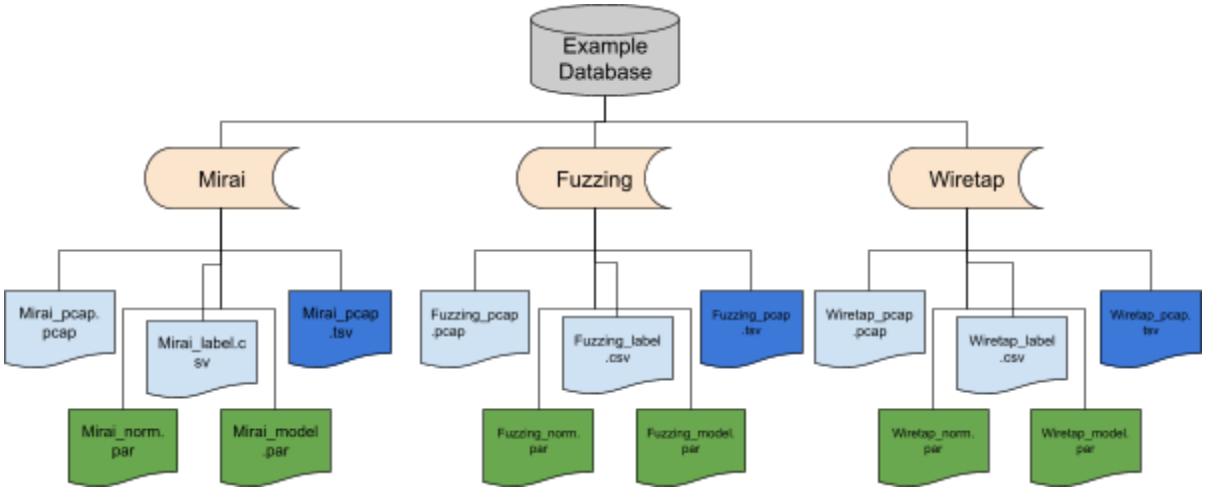
The pcap examples file directory contains several examples of malicious network traces. At the top level, the directory has a folder for each type of malicious activity: Mirai, Fuzzing, and Wiretap. Under each activity directory, file names are standardized, so that the user/developer can specify just the name of the attack and all other file names are generated automatically.

Each directory contains the full pcap recording and its associated label file. These source files are shown in light blue in the directory diagram below. File names are <activity>_pcap.pcap and <activity>_label.csv. The Feature extraction module generates a matching tab separated value file using tshark, which is stored in this directory under file name <activity>_pcap.tsv. This derived file is shown in blue.

The pcap data files have mixed normal and malicious packet records. The label file indicates which records are benign and which are malicious. A section of all benign data at the beginning of each file is used for training. The application parses the label file to automatically determine what part of the file should be used for the selected mode: training or execution. We include a startup grace period at the beginning of the execution data to keep the statistics well behaved and consistent with Kitsune results.

Each directory also includes the normalization parameters for each feature, and the trained NN model (weights, biases, and map). Files names will be <activity>_norm.bin and <activity>_model.bin. These trained model files are shown in green in the directory diagram. Format of these files is discussed in the KitNET model section below. The normalization parameters are stored in this directory, since they are used by the Feature Extraction module.

The application also creates a temporary directory in the user's current working directory for storing files that pass between the processes. The cryptographic context is stored in crypto_context.bin and the evaluation key in relin_key.bin. The batched and encrypted data files are written and read from this directory with file names: feature_ciphertext_<fn>.bin, where "fn" is the feature number, starting with 0. The encrypted score file is also stored here in score_ciphertext.bin. Access to the files is coordinated using a communication pipe between the client and server processes. Only one batch of data will appear in the directory at a time. The python main script will be responsible for clearing and deleting this directory before terminating.



Feature Extraction Module

The feature module is derived from the Feature Extraction package in Kitsune. This module is used for both training and client processes. The Feature Extraction module is based on the *FeatureExtractor*, *NetStat*, and *IncrStat* classes from fhe-Kitsune. We expect no significant changes to operation. These classes are already encapsulated in their own package.

The feature extraction class creates a dictionary of different traffic connections, keyed by source IP : destination IP. There are two entries per connection, one each direction. On receiving a new

packet, it determines which connection it belongs to, and then accumulates incremental statistics for that dictionary entry. It also extracts information from the companion entry to compute cross correlation statistics. The module then computes and returns a set of feature values for that packet based on the current statistics. It produces a feature vector for every packet.

Kitsune uses the Wireshark utility, tshark, to extract relevant fields from the pcap files, into a more manageable tab separated value file. We continue to use that approach in this application. Wireshark/tshark must be installed on any machine using the application. One option for speeding operation is to look for an existing tsv file before activating tshark to convert the file.

KitNet Module

The KitNet model encapsulates the trained neural network, along with the training, in-the-clear execution, and encrypted execution functions. This module is derived from the Kitsune KitNET package and uses the same classes: *KitNET* and *dA*. The current Kitsume examples move directly from training to execution, allowing all parameters to be stored internally. Since our FHE application uses separate training and execution processes, parameters must be written to storage files after training for use by later processes. In addition, not all of the KitNET mathematical operations are homomorphic in FHE, so approximations have been developed.

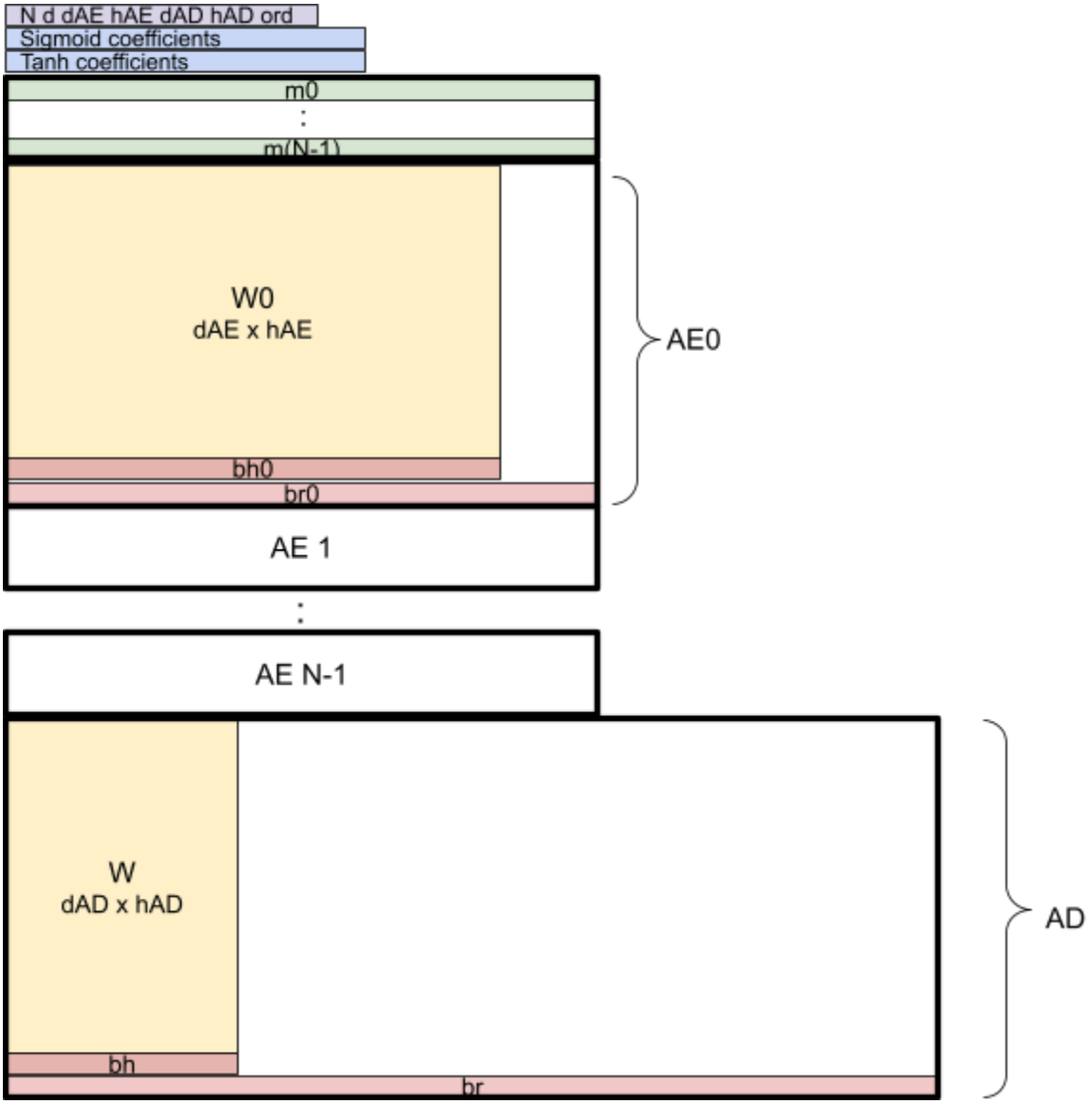
KitNET model files

The KitNet module is derived from the Kitsune KitNET package. Thus to support separate training and execution processes, as well as separate client and server processes required some changes. To pass model parameter information between the training and execution, means that the model parameters is written and read from a model parameter file. Similarly, trained normalization parameters are stored in a parameter file. Read and write functions are defined within the classes to access and store this parameter information. The python `struct` package is used to write these binary files to give us the flexibility to change the structure to suit the FHE implementation as it is designed. The python package `pickle` is not used in our application due to security weaknesses of that package.

The planned layout of the NN parameter file is shown in the diagram below. The first entry is a header, which contains:

- the number of autoencoders (AE), N,
- the feature vector dimension, d,
- the input dimension for all autoencoders, dAE,
- the hidden dimension for all autoencoders, hAE,
- the input dimension for the anomaly detector, dAD,

- the hidden dimension for the anomaly detector, hAD, and
- the order of the logistic approximations, ord



The header is followed by the coefficients for the sigmoid and tanh Chebyshev approximation functions and then the feature to autoencoder maps, each of which lists the feature indices of the inputs to the respective autoencoder. The parameters for the N autoencoders follow the feature maps. Each autoencoder has the following parameters

- The transform Wi, which has dimension dAE x hAE $(row\ x\ col)$.

- The bias for the hidden layer, bhi, which has dimension 1 x hAE.
- The bias for the reconstruction, bri, which has dimension 1 x dAE.

The N sets of autoencoder parameters are followed by the anomaly detector parameters.

- The transform W, which has dimension dAD x hAD.
- The bias for the hidden layer, bh, which has dimension 1 x hAD.
- The bias for the reconstruction, br, which has dimension 1 x dAD.

Integer values will be 2 byte shorts and floating point values will be 8 byte doubles.

The normalization parameter file is much simpler, since it contains only the standard deviation and mean of each feature. It is structured as a two column array with d entries. The i^{th} row contains first the offset followed by the scale for the i^{th} feature.

KitNET FHE modifications

Limitations of FHE libraries also necessitate changes to KitNET. A fixed NN architecture simplifies operation and conversion to FHE processing. In addition, FHE is not homomorphic in the NN non-linear activation functions, so polynomial approximations to these functions were developed.

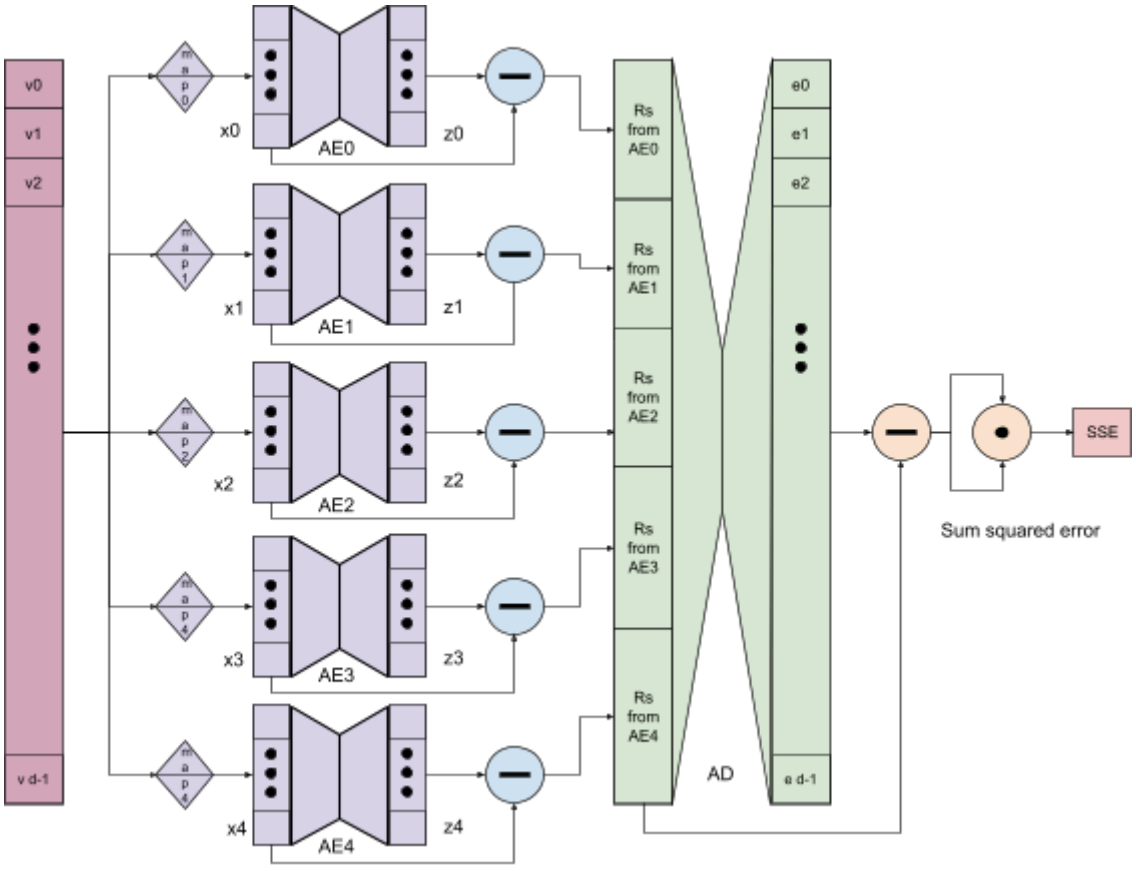
Model Architecture

Numerous trials on different network activity data sets shows that the number of autoencoders and the partition of features to autoencoders varies little. Typically the same statistics from all the different time filters are grouped together. In addition, closely related statistics, such as mean and magnitude or covariance and Pearson correlation coefficient, are also grouped together. Most solutions have five to seven autoencoders. Those with more than five typically have one or two autoencoders with a single feature. These isolated features are likely a side effect of the partitioning algorithm. Otherwise, the feature partition is very consistent from one data set to the next. Consequently, we fix the model architecture to five autoencoders (AEs) with feature assignment learned on the Mirai data.

AE 0 uses all the packet count statistics for both packet length and jitter. Jitter is the time between packet transmissions. AE 1 uses all the mean and magnitude packet length statistics. AE 2 uses the variance and radius packet length statistics. AE 3 uses the mean and variance of the packet jitter. AE 4 uses the packet length covariance and Pearson correlation coefficient statistics.

The resulting NN execution flow is shown in the above figure. The full dimension (d) feature vector, v, is input to the NN ensemble. Each autoencoder has a map, which specifies the features

that are used by that autoencoder, as well as to which nodes they are connected. Each autoencoder operates on its own features vector x, producing a reproduced feature vector, z. The difference or residual, r, between these vectors is then computed. The residual values are stacking into a new full dimension vector. The anomaly detector operates on this d-dimensional r

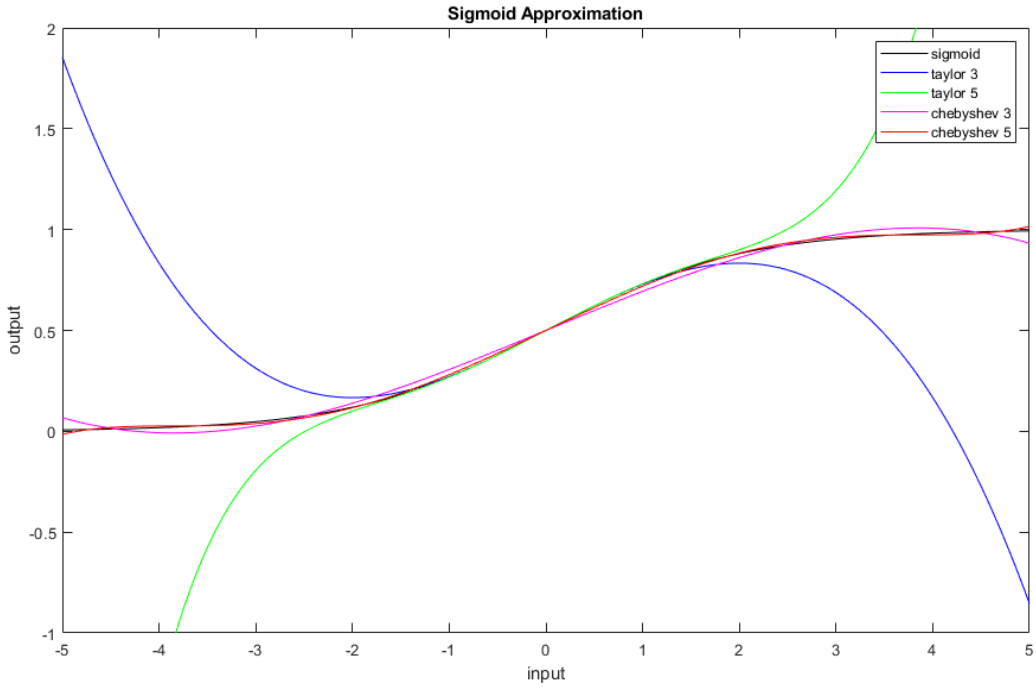


vector, producing a reproduced error vector, e. The difference between the residual and reproduced error vector is computed, and the dot product taken to produce a sum squared error scalar output.

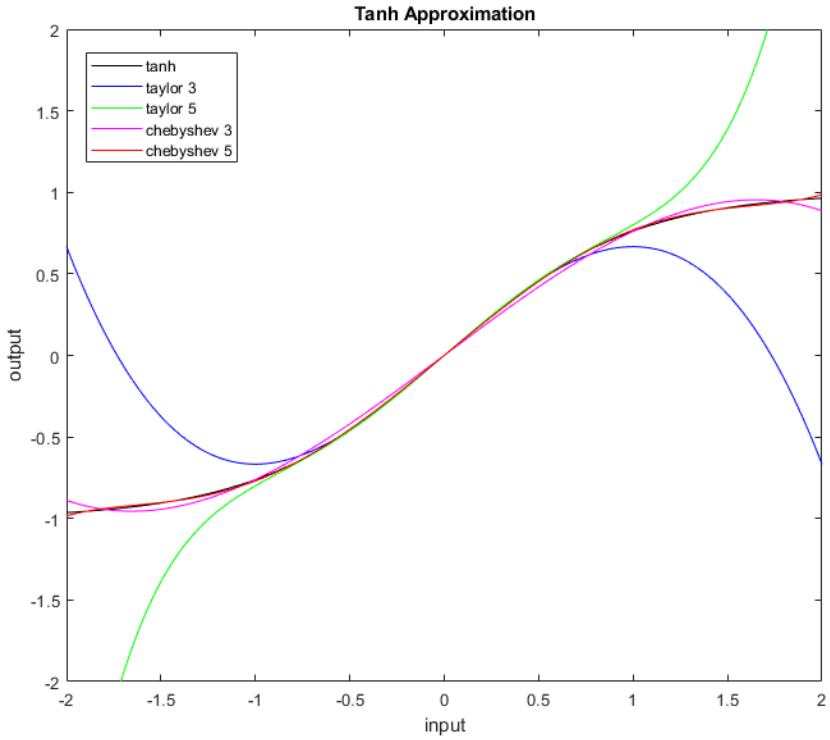
Logistic approximation

FHE is only homomorphic in addition and multiplication. Consequently, it can not directly implement the logistic functions needed for neural networks. Instead, we use Chebyshev polynomial approximations for the logistics in the execution process. For client-side model training, we use *sigmoid*, $1/(1 + e^{-x})$, and *tanh* mathematical functions from the numpy library directly.

We use Chebyshev polynomials, since they have the advantage of fitting between two extremes, unlike Taylor series expansions that are fit around a single point, and have limited dynamic range before they diverge strongly. Analysis of Kitsune on several example data sets indicates that the inputs to the autoencoder *sigmoids* are within the range of +/- 5. A 5th order polynomial fit produces good results, although a 3rd order polynomial may be feasible and will reduce the multiplicative depth by two. (Logistic functions are predominately odd, so a 4th order fit is no



better than a 3rd order.) The inputs to the anomaly detector *tanh* functions are within the range of +/- 2. A 5th order polynomial provides good results. The accuracy of Chebyshev and Taylor series polynomial approximations are shown in the plots.



FHE Execution of KitNET

This section describes the implementation of anomaly detection in the FHE environment and within the server process. We first describe the inputs to and outputs from the server, followed by the order of operations. For a flowchart of the server, see the architecture section above. This section ends with a description of the FHE operations and processing.

Inputs

The server takes several inputs, some related to the cryptographic context, others to the neural network model, in addition to the encrypted feature data. Crypto context data includes both the FHE cryptographic parameters and the evaluation keys. Model parameters define the NN weights and biases, as well as activation function approximation parameters. These parameters are delivered to the server as already described. Encrypted batches of features are sent periodically to the server for processing, also as already described. The full list of inputs is given below:

- Crypto context: an OpenFHE CKKS CryptoContext cryptocontext : openfhe.CryptoContext preconfigured on the client (depth, batch size, scale, etc.), serialized by the client and deserialized on the server.
- Evaluation Keys:
    - EvalMultKey**:** for multiplications.
    - EvalAutomorphismKeys**:** for rotations, which seem optional in our case.
- Model parameters (cleartext) for k = 0..N-1 autoencoders and one final anomaly detector (indexed as N):
    - m[k] : List[int] (length d[k]): feature indices into Ef
    - Wenc[k] : List[List[float]] shape (h[k], d[k])
    - Wdec[k] : List[List[float]] shape (d[k], h[k]). This is the transpose of Wenc.
    - bh[k] : List[float] length h[k]
    - bv[k] : List[float] length d[k]
    - Sigmoid approximation: coeffs_sigma, bounds_sigma = (aσ, bσ) and
    - Tanh approximation: coeffs_tanh, bounds_tanh = (at, bt)
- Encrypted batch of features: Ef : List[openfhe.Ciphertext] with length F (e.g., F = 50).

  Packing layout (full CKKS packing):

    - Let N = cc->GetRingDimension() and B = N/2 be the number of slots. We consider B = 32,768 (when N = 65,536).
    - Each Ef[f] encrypts feature f across all B packets: for packet index s ∈ [0, B-1], slot s of Ef[f] holds the value of feature f for packet s.
    - There is one feature per ciphertext, slots do not mix different features.

Outputs

The server has a single output: an encrypted vector of anomaly scores. The vector is the same length as the input ciphertexts and each slot holds the score for the corresponding input slots.

- Score_ct : openfhe.Ciphertext: each where slot s is the anomaly score for packet s.

Server Order of Operations

The server has a fairly simple structure. At startup it loads the context, keys, and model. It encodes the model parameters, so that they can be used by OpenFHE or Fideslib (OpenFHE compatible GPGPU functions). Once initialized it waits for a set of ciphertexts. When they are received it reads them, and processes them through the model, creating a score ciphertext. After writing that ciphertext, it goes back to waiting for another input. In summary, the server flow is:

1. Deserialize cc and evaluation keys; enable PKE, KEYSWITCH, and LEVELEDSHE features (matching the client).
2. Load the cleartext model parameters and encode them using the crypto context
3. Load the encrypted feature set Ef[0 ... F-1] and the encoded model parameters.
4. Execute the full pipeline under encryption (all autoencoders + final detector), strictly slot-wise, leveraging the feature-per-ciphertext packing.
5. Serialize and return encrypted Score_ct.

Server initialization (deserialize context and keys)

This stage loads the client-generated context, enables required features, and brings in the evaluation keys.

```python
import openfhe

cryptocontext = openfhe.DeserializeFromFile(filename1, openfhe.SERBINARY)
cryptocontext.Enable(openfhe.PKE)
cryptocontext.Enable(openfhe.KEYSWITCH)
cryptocontext.Enable(openfhe.LEVELEDSHE)
cryptocontext.DeserializeEvalMultKey(filename2, openfhe.SERBINARY)
```

Homomorphic operations

The OpenFHE operations used in the NN model execution are:

- Add/Sub: cryptocontext.EvalAdd, cryptocontext.EvalSub (slot-wise)
- Mul by scalar: cryptocontext.EvalMult(ct, float) (preferred)
- Bias add: cryptocontext.EvalAdd(ct, float)
- ct*ct mul (for square): cryptocontext.EvalMult(ctA, ctB) (needs EvalMultKey)
- Chebyshev activations: cryptocontext.EvalChebyshevSeries(ct, coeffs, a, b)

Pseudocode (FHE + NN Execute):

The execution stage runs all autoencoders to build residuals, followed by the final detector to produce the encrypted per-slot score. This algorithm design leverages the ensemble structure of the autoencoder neural network to reduce computational complexity.

```python
def fhe_execute_autoencoder_ensemble(cryptocontext, Ef, model):
```



```
    N  = model["N"]
    d  = model["d"]
    h  = model["h"]
    m  = model["m"]
    Wenc = model["Wenc"]
        bh   = model["bh"]
    bv   = model["bv"]


    def lin_comb_cipher_const(X_row, w_row, b_scalar):
        A = cryptocontext.EvalMult(X_row[0], w_row[0])
        for j in range(1, len(X_row)):
            A = cryptocontext.EvalAdd(A, cryptocontext.EvalMult(X_row[j],
w_row[j]))
        return cryptocontext.EvalAdd(A, b_scalar)

    def act_sigma(ct):
        a, b = bounds_sigma
        return cryptocontext.EvalChebyshevSeries(ct, coeffs_sigma, a, b)

    def act_tanh(ct):
        a, b = bounds_tanh
        return cryptocontext.EvalChebyshevSeries(ct, coeffs_tanh, a, b)

    R_ct = []

    for kk in range(N):
        X_ct = [Ef[m[kk][jj]] for jj in range(d[kk])]

        Y_ct = []
        for ii in range(h[kk]):
            A = lin_comb_cipher_const(X_ct, Wenc[kk][ii], bh[kk][ii])
            Y_ct.append(act_sigma(A))

        for jj in range(d[kk]):
            w_row = [Wenc[kk][ii][jj] for ii in range(h[kk])]
            A = lin_comb_cipher_const(Y_ct, w_row, bv[kk][jj])
            Z = act_sigma(A)
            R_ct.append(cryptocontext.EvalSub(X_ct[jj], Z))
```



```
    YN_ct = []
    for ii in range(h[N]):
        A = lin_comb_cipher_const(R_ct, Wenc[N][ii], bh[N][ii])
        YN_ct.append(act_tanh(A))

    Score_ct = None
    for jj in range(d[N]):
        w_row = [Wenc[N][ii][jj] for ii in range(h[N])]
        A = lin_comb_cipher_const(YN_ct, w_row, bv[N][jj])
        E = act_tanh(A)
        diff = cryptocontext.EvalSub(R_ct[jj], E)
        sq = cryptocontext.EvalMult(diff, diff)
        Score_ct = sq if Score_ct is None else cryptocontext.EvalAdd(Score_ct, sq)

    return Score_ct
```

Memory and Complexity

Rough memory use assessment (pre-implementation): With CKKS at ring dimension 2^{16} and depth ~ 21, assuming about 30 RNS limbs in the chain, a ciphertext has 2 components and is about 2 * 65536 * 30 * 8 bytes, which is ~ 30 MB. Each ciphertext packs one feature across 32,768 slots (one per packet), so keeping all 50 feature ciphertexts resident is ~ 1.5 GB. If we kept everything in memory during execution, we would also hold a residual vector of similar size (approx 1.5 GB) plus a small set of intermediate ciphertexts (e.g., few hundred MB), putting the peak working set around to about 3 GB. However, because we execute per ciphertext/autoencoder rather than all at once, the peak drops substantially. The relinearization key (EvalMultKey) is typically a few tens of MB. Given our runs with inputs normalized to (0,1) and FLEXAUTO rescaling, these order-of-magnitude figures are representative at depth ≃ 21.

Rough computational complexity assessment (pre-implementation): per batch we perform about $2.\Sigma_k h[k].d[k] + 2.h[N].d[N]$ ciphertext-scalar multiplies, d[N] ciphertext-ciphertext multiplies (for final squaring), and four activation (Chebyshev) per unit; all operations run SIMD over 32,768 slots.

Batched data module

This module gathers up the features/statistics from a “batch” of network traffic packets for encryption and processing. Feature extraction continues to gather incremental statistics, since

that approach effectively highlights changes in the network, as well as minimizing temporary data storage. At the same time, our goal is to pack the ciphertext as full as possible, to amortarize the encryption data growth over as many packets as possible. The purpose of this module is to gather up and format the input data into batches to be processed by the FHE software. Each "batch" of data consists of 32K packets. Data is delivered to the Crypto Context module as a list of 50 lists, one sublist per feature. The sublists are lists of python 'floats' and contain the corresponding feature values for all 32K packets.

This module includes functions for serialization to a file and deserialization from a file for the in-the-clear data and scores. Reading and writing encrypted data is the responsibility of the cryptographic context module (next section). For consistency with OpenFHE, we use a separate file for each feature, which corresponds to a ciphertext. File names are tagged with the feature index to enable automated reading and organization by the server.

Cryptographic context module

The cryptographic context module defines and maintains the cryptographic "context" for FHE operation. It defines necessary cryptographic parameters, generates keys, encodes and encrypts the input data. It is also responsible for decrypting and decoding encrypted data. Initially, this module includes deserialization/read and serialization/write to a set of files. These files include the crypto context, the processing key, as well as one file per feature of input data.

The crypto context and keys are generated at client startup. This data is passed to the server via the file system. Thereafter, the client gathers batches of 32K packets and converts them into ciphertexts. These ciphertexts are passed to the server for processing, also through the file system. The client receives as a result of the server computation a score ciphertext, which it decrypts and deliver to the user interface for plotting.

Generating the cryptographic context

This section establishes the CKKS parameters to support the server computation, and informs generation of the OpenFHE CryptoContext object.

OpenFHE chooses the CKKS parameters based on the basic properties we desire the parameters to support, minimally including multiplicative depth, scale factor, and batch size. Based on our earlier work, we expect the multiplicative depth to be at least 22, scaling modulus size to be at least 54, and the batch size to be 32768. For example:


```python
import openfhe

parameters = openfhe.CCParamsCKKSRNS() # Create a CKKS parameters object base
parameters.SetSecretKeyDist(openfhe.UNIFORM_TERNARY)
parameters.SetSecurityLevel(openfhe.HEStd_128_classic)
parameters.SetMultiplicativeDepth(multiplicative_depth)
parameters.SetScalingModSize(scale_modulus_size)
parameters.SetScalingTechnique(openfhe.FLEXIBLEAUTO)
parameters.SetBatchSize(batch_size)
...
```

Once the parameters object is complete, it is passed to the GenCryptoContext function to create the cryptography context. Once that object is created, several other scheme-level features (PKESchemeFeature) can be set. OpenFHE is not clear on why certain features get set in the parameters object, while others get set in the CryptoContext object. For example:

```python
cryptocontext = openfhe.GenCryptoContext(parameters)
cryptocontext.Enable(openfhe.PKE)
cryptocontext.Enable(openfhe.KEYSWITCH) # Support the standard evaluation keys
cryptocontext.Enable(openfhe.LEVELEDSHE)
...
```

The CryptoContext object contains all the FHE-related methods, including key generation, encoding and decoding, encryption and decryption. Once the client creates the CryptoContext object, it can serialize it to be sent to the server.

```python
openfhe.SerializeToFile(filename, cryptocontext, openfhe.SERBINARY)
```

Note that the server can use the CryptoContext *alone* – more precisely, without receiving any evaluation keys or ciphertexts – to encode any desired cleartext data as CKKS plaintexts. Depending on the size of the model data, it may be advantageous to have the server encode the model as soon as it receives and deserializes the CryptoContext, so that it is ready to start computing as soon as it receives and deserializes the client ciphertexts.

Key generation

Once the client has generated the CryptoContext, it can generate its private key and the public encryption key, along with any evaluation keys it needs. Once generated, the evaluation keys are stored in the CryptoContext. At present, we use only a relinearization key. For example:

```
Python
key_pair = cryptocontext.KeyGen() # Generates the public/private key pair

"""Use the private key to generate the required evaluation keys, then
serialize to file."""
cryptocontext.EvalMultKeyGen(key_pair.secretKey) # relinearization key
cryptocontext.SerializeEvalMultKey(filename, openfhe.SERBINARY)
```

OpenFHE manages all the seeding and randomness generation internally, so the library user never needs to interact with it directly, beyond selecting the desired secret key distribution when generating the parameters. However, the choice of that distribution is itself dictated by the FHE security guidelines.

Creating ciphertexts from cleartext data

Once the client has generated the required keys, it can move on to encoding and encrypting the batched data. The MakeCKKSPackedPlaintext function takes a list of floats and creates a CKKS plaintext, which can then be encrypted using the Encrypt method:

```
Python
cleartext = [3.14159, 1, 1, 2, 3, 5, 2.718281828] # example list of floats
plaintext = cryptocontext.MakeCKKSPackedPlaintext(cleartext)
ciphertext = cryptocontext.Encrypt(key_pair.publicKey, plaintext)
openfhe.SerializeToFile(filename, ciphertext, openfhe.SERBINARY)
```

Note that Python's float type is dependent on the underlying system double type, which may vary across the client and server systems.

The size of the serialized ciphertexts will depend on the final CKKS parameters chosen. Based on our initial estimates for the multiplicative depth (22) and batch size (32768), we expect the raw ciphertext polynomials to take up:

(23 primes)*(8 bytes per coefficient)*(65536 coefficients per prime per polynomial)*(2 polynomials per ciphertext) = 24,117,248 bytes = 24.11725 MB

Encrypting 50 parameters per packet with 32768 packets per batch gives us a total of 50 ciphertexts, for which the total ciphertext storage required will be at minimum 1.205863 GB.

Decrypting result ciphertexts

OpenFHE helpfully combines decryption and decoding into a single function call, so when the client receives the result ciphertext, all it needs to do is the following:

```python
result = cryptocontext.Decrypt(key_pair.secretKey, ciphertext)
```

References

1. Mirsky, Yisroel; Doitshman, Tomer; Elovici, Yuval; Shabtai, Asaf. “Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection”. NDSS ‘18, Network and Distributed Systems Security Symposium, 1802.09089, 2018.
2. Cheon, Jung Hee; Kim, Andrey; Kim, Miran; Song, Yongsoo. “Homomorphic encryption for arithmetic of approximate numbers.” ASIACRYPT 2017, 23rd international conference on the theory and applications of cryptology and information security, 409–437, 2017.

Appendix A: Network Intrusion Detection

The Kitsune open source software package provides a proven on-line network intrusion detection (NID) baseline. It has a light-weight anomaly detection (AD) structure, which allows it to be trained using ordinary network traffic and deployed to operate in real time on live networks. Its architecture consists of an ensemble of autoencoders, whose multiply-add characteristics are a good match to FHE capabilities. This simple mathematical structure makes Kitsune a viable FHE application, unlike complex deep neural network (DNN) intrusion detection.

The original Kitsune package is described in Mirsky, Yisroel; Doitshman, Tomer; Elovici, Yuval; Shabtai, Asaf. “Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection”. NDSS ‘18, Network and Distributed Systems Security Symposium, 1802.09089, 2018. Python based source code is available on GitHub at https://github.com/ymirsky/KitNET-py.

NID Algorithm Description

Kitsune intrusion detection algorithm consists of three primary parts: feature extraction, model training, and anomaly detection. Feature extraction creates a feature vector of relevant statistics for every packet in a stream of packets. Model training uses known normal traffic to train the Neural Network based anomaly detection. Anomaly detection applies the trained model to the feature vector and produces an anomaly score. The higher the score the greater the probability of an anomalous packet. A connection or time period with “enough” anomalous packets indicates an attempted intrusion. Kitsune does not set intrusion detection thresholds, but leaves that to the user, as different users have greater or lesser tolerance for false alarms and missed detections.

Feature Extraction

Kitsune feature extraction uses an incremental statistics approach, which accumulates first and second order statistics for each connection, where connections are specified by Internet Protocol (IP) or Media Access Control (MAC) source and destination addresses. Higher level protocols, such as TCP, UDP, TLS, use IP addresses to identify connections and lower-level protocols, such as ARP, use MAC addresses. To capture both short and long term behavior, statistics are calculated over five different temporal filter lengths. These are low-pass filters, which have similar behavior to a moving average, but without the overhead required to store past values. Kitsune uses five different filters with 3 dB points of 0.2 s, 0.33s, 1 s, 10 s, and 100 s.

The feature statistics per packet are:

- Number of packets in packet length calculations

- Mean packet length
- Packet length variance calculated from residuals
- Number of packets in jitter (inter-transmit time) calculations
- Mean jitter
- Jitter variance calculated from residuals
- Packet length “magnitude” (squareroot of sum of means squared for input and output)
- Packet length radius (sum of variances for input and output)
- Cross-correlation between output and input packet lengths
- Pearson correlation coefficient (cross-correlation normalized by standard deviations)

The total number of features calculated per packet is 3 statistics * 5 filters * 2 inputs (packet length and inter-transmit time) + 4 statistics * 5 filters * 1 input (packet length) = 50.

Packet lengths are usually less than 14 bits, and time differences are on the order of a few milliseconds to a few seconds and can be represented in 12 bits. This suggests that acceptable results could be achieved using 16 bit fixed point input values. Since values have been normalized to be between 0 and 1, the binary point will be on the left.

Model Training

Default KitNET model training is used in our application. The first 10% of the training data is used to determine normalization parameters and autoencoder architecture. The remaining 90% of the training data determines the parameters for the autoencoder and anomaly detector neural networks.

Every value must be normalized prior to encryption to a floating point value (double) scaled between 0 and 1. The baseline implementation of Kitsune assumes a linear scaling based on an estimate of maximum and minimum values. This approach leads to inputs that can be outside the 0 to 1 dynamic range; this is especially true of anomalous network packets. Consequently, a non-linear scaling, such a sigmoid parameterized by expected mean and variance, will be used to ensure that the output dynamic range is strictly limited.

To train the normalization parameters, feature values are first computed. Mean and variance statistics for each feature value are measured over the initial training set. Mean is used as the offset, and the square root of the variance, the standard deviation, is used to define the scale. Using a scaling of 4 times the standard deviation produced good results.

$$\hat{x} = \frac{1}{1+e^{-(x-\mu)/4\sigma}}$$

The autoencoder architecture, that is, the number of autoencoders and assignment of features to autoencoders, is also learned from the initial training set. The correlations between all feature pairs are computed at the same time that the mean and variance statistics are gathered. Incremental hierarchical clustering using these correlations as the clustering metric, is performed to organize the features into similar sets. A maximum cluster size prevents the process from degrading into a single cluster. Using a maximum cluster size of twice the number of time scales produces consistently sensible architectures.

Different sets of network traffic data produce very similar architectures. Differences are largely due to one or two outliers that do not cluster as expected. Statistics for each time filter are always grouped together. In addition, similar statistics, such as covariance and Pearson correlation coefficient, or mean and magnitude, are also grouped together.

Once the number of autoencoders and feature assignments are learned, the autoencoder parameters are initialized to random values. Each feature vector is normalized and then used to perform a single optimization pass on the autoencoder parameters, after which it is discarded. Autoencoder weights and biases are optimized to reproduce the input data with minimal mean squared error over the set of training data. The error between input and output of the ensemble of autoencoders drives a final anomaly detector, which also has the form of an autoencoder. Weights and biases for this autoencoder are also optimized to reproduce the input data with minimal mean squared error. The resulting neural network produces large anomaly scores when the input data does not match the “normal” traffic patterns and the error pattern also does not match typical patterns.

Anomaly Detection

The KitNet anomaly detection engine within the Kitsune package largely consists of multiply-adds operations, making it a possible application within FHE processing limitations. The first stage of the KitNet anomaly detector consists of an ensemble of k auto-encoders. Each autoencoder operates on a subset of the full feature vector of dimension d. The output is the difference between the input and reconstructed values. Each autoencoder is described by an n x m matrix of weights, W, and two bias vectors, b_m, and b_n, where n is the dimension of the input subvector, x, and m is the dimension of the autoencoder’s hidden layer. For KitNet, the hidden layer is set to ¾ of the input dimension, rounding up. Each autoencoder includes two sigmoid functions, indicated in the equations and is defined as

$$\frac{1}{1+e^{-a}}$$

The autoencoder operation is:

$$\widetilde{x} = \sigma\left(\sigma\left(x \cdot W + b_m\right) \bullet W^T + b_n\right)$$

where $\widetilde{x}$ is the reconstructed vector. The difference or error is

$$\varepsilon = \left(x - \widetilde{x}\right)$$

The errors from all autoencoders are combined into a new vector, e, of dimension, d, which is then sent to a final anomaly detector autoencoder. This autoencoder also has a transform matrix and two bias vectors with hidden dimension, k, the number of first stage autoencoders. Since the input ranges from -1 to +1, this neural network uses a hyperbolic tangent (tanh) logistic function. It produces a mean squared error value (scalar) from the difference between the input and output vectors. This output value is linear on a logarithmic scale, with larger values indicating more severe anomalies.

